

PC
GAME

ELIGE UN NÚMERO DEL 1 al 100

UN NÚMERO,
INFINITAS POSIBILIDADES



ESTRATEGIA
Y SUERTE



PARA TODAS
LAS EDADES



DESAFÍA A
TUS AMIGOS



DIVERTIDO,
FÁCIL Y ADICTIVO

UIDE

POWERED BY ARIZONA STATE UNIVERSITY

Proyecto Integrador de Lógica de Programación

Diseño y Análisis del Juego «Adivina el Número» — Análisis funcional, arquitectura de software y estructuras de datos en Python.



Análisis Funcional

Descripción del problema, justificación y objetivos del sistema.



Arquitectura de Software

Arquitectura modular, mantenibilidad, eficiencia y escalabilidad.



Estructuras de Datos

Integración funcional de tuplas, listas y diccionarios en Python.

Estudiante

Mery Elizabeth
Guzmán Ontaneda

Docente

MCS. Lilian Aman

Materia

Lógica de
Programación

Fecha

Junio 22–28, 2026

Introducción y Desarrollo del Tema

La programación constituye una forma de pensamiento estructurado que permite resolver problemas de manera eficiente y reproducible. Este proyecto, correspondiente a la Semana 8 de Lógica de Programación (UIDE), implementa el juego «Adivina el Número», donde la computadora identifica un número del 1 al 100 usando búsqueda binaria con un máximo garantizado de siete intentos.

La versión final incorpora tres estructuras de datos fundamentales —tuplas, listas y diccionarios— con roles semánticos precisos, no como ornamento, sino demostrando uso real durante la ejecución. La búsqueda binaria, con complejidad $O(\log_2 n)$, garantiza matemáticamente que para 100 números nunca se necesitarán más de siete intentos.

R1 — Validación

Acepta únicamente respuestas válidas del usuario.

R2 — Búsqueda Binaria

Algoritmo sin alteraciones en su lógica fundamental.

R3 — Terminación

Garantizada por victoria o agotamiento de intentos.

R4 — Presentación

Trayecto completo de intentos al finalizar la partida.

Herramientas y Análisis de Software

Stack Tecnológico

Python 3.14 — Tipado dinámico con anotaciones de tipo opcionales (PEP 484). Estructuras nativas: tuple, list, dict.

VS Code 1.110 — IntelliSense, depuración interactiva, terminal integrada, verificación de tipos con Pylance.

RAPTOR 4.1.0 — Diseño algorítmico visual mediante diagramas de flujo ejecutables paso a paso.

Arquitectura y Calidad

El sistema implementa una **arquitectura monolítica modular de tres capas**: presentación (I/O con usuario), negocio (búsqueda binaria en `jugar()`) y datos (historial y estadísticas en memoria).

La **complejidad temporal** es $O(\log_2 n)$: para $n=1.000.000$, la búsqueda lineal necesitaría 1.000.000 intentos; la binaria, apenas 20. La **complejidad espacial** es $O(1)$ — el historial almacena como máximo 7 diccionarios de tamaño constante.

La **escalabilidad** permite ampliar el rango, agregar respuestas, persistir datos o construir una interfaz gráfica sin modificar el núcleo algorítmico.

Resultados, Implicaciones y Limitaciones

El programa cumple los cuatro requisitos funcionales: valida entradas, identifica cualquier número en máximo 7 intentos, termina siempre y presenta el trayecto completo. La traza para el número 37 confirma victoria en 3 iteraciones —frente a 7 garantizados—, demostrando escalado óptimo.

Implicaciones Pedagógicas

Integración funcional —no cosmética— de estructuras de datos. El estudiante ejerce pensamiento crítico al elegir la herramienta correcta según las propiedades del dato.

Implicaciones de Diseño

«Código que comunica intención»: el tipo de dato revela la naturaleza del dato. Las tuplas comunican fijeza; las listas, crecimiento; los diccionarios, acceso semántico.

Limitaciones Identificadas

Sin verificación de honestidad del usuario, interfaz exclusivamente de consola, volatilidad de datos en memoria, ausencia de manejo formal de excepciones y de pruebas unitarias automatizadas.

Estructura Inicial del Código

El proyecto está diseñado con una arquitectura clara y modular, organizando el código en componentes lógicos que facilitan la comprensión, el mantenimiento y la escalabilidad del juego.

1

Constantes de Módulo

Define valores fijos e inmutables, como `RANGO` para los límites del juego y `RESPUESTAS_VALIDAS` para las interacciones del usuario. Esto asegura la coherencia y evita "números mágicos" dispersos en el código.

2

Función `jugar()`

Encapsula toda la lógica principal del juego, desde la inicialización del estado hasta la gestión de los intentos y el algoritmo de búsqueda binaria. Promueve la reusabilidad y facilita las pruebas unitarias.

3

Bloque de Ejecución Principal

Identificado por `if __name__ == "__main__":`, este bloque es el punto de entrada del programa. Se encarga de llamar a la función `jugar()` e iniciar el flujo de la aplicación.

Esta organización promueve la legibilidad y la adherencia a buenas prácticas de ingeniería de software, como la separación de responsabilidades y la modularidad.

Ciclos de Programación en el Juego

El juego "Adivina el Número" se basa fundamentalmente en estructuras de control de flujo para su lógica, garantizando la interactividad, la validación de entradas y la ejecución del algoritmo de búsqueda binaria.



Ciclo Principal (**while**)

Un bucle **while** controla las rondas del juego, continuando hasta que se adivina el número o se agotan los intentos. Permite la iteración constante de conjeturas y retroalimentación.



Condicionales (**if/else**)

Las sentencias **if**, **elif** y **else** dirigen la lógica de la búsqueda binaria, ajustando el rango de búsqueda (mayor/menor) o declarando la victoria (correcto).



Validación de Entrada (**while**)

Un bucle **while** anidado asegura que el usuario proporcione una respuesta válida ('mayor', 'menor', 'correcto'), pidiendo repetidamente la entrada hasta que sea aceptable.

Estructuras de Datos y Organización del Código

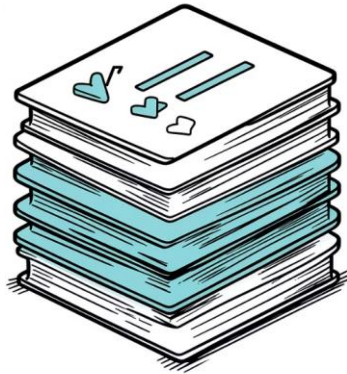
La incorporación de estructuras de datos enriquece la gestión de la información: consolida constantes inmutables, registra el trayecto cronológico y estructura la información de cada intento y sesión.



Tuplas — Inmutables

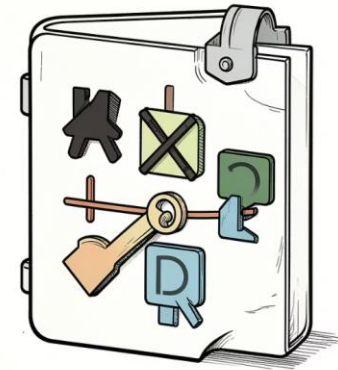
RANGO = (1, 100): límites inmutables del espacio de búsqueda. **RESPUESTAS_VALIDAS** = ('mayor', 'menor', 'correcto'): conjunto fijo de entradas aceptadas.

rango_snapshot: captura inmutable del estado del rango en cada conjetura, preservando el historial exacto.



Listas — Dinámicas

historial: colección mutable y ordenada que crece con `.append()` en cada iteración. Refleja el orden cronológico de intentos. Cada elemento es un diccionario, ilustrando el patrón «lista de diccionarios» para representar tablas de datos.



Diccionarios — Semánticos

registro: datos de cada intento con claves nombradas. **estadísticas**: métricas acumuladas de sesión. **Retorno de jugar()**: diccionario con cuatro claves que evita variables globales y comunica resultados por nombre.

El código se organiza en tres bloques: constantes de módulo, función `jugar()` y bloque `__main__`, siguiendo principios de modularidad, separación de responsabilidades y documentación integrada mediante anotaciones de tipo y docstring.

Conclusiones y Recomendaciones

1 Búsqueda binaria óptima garantizada

La complejidad $O(\log_2 n)$ asegura máximo 7 intentos para $n=100$, verificado experimentalmente.

2 Estructuras no intercambiables

Tuplas para constantes inmutables, listas para colecciones dinámicas, diccionarios para acceso semántico por clave.

3 Modularidad y arquitectura

Encapsular lógica en `jugar()` sin variables globales; tres capas permiten evolución independiente de presentación, lógica y datos.

4 Documentación como calidad

Docstring, anotaciones de tipo y comentarios sobre el «por qué» hacen el código comprensible para cualquier programador.

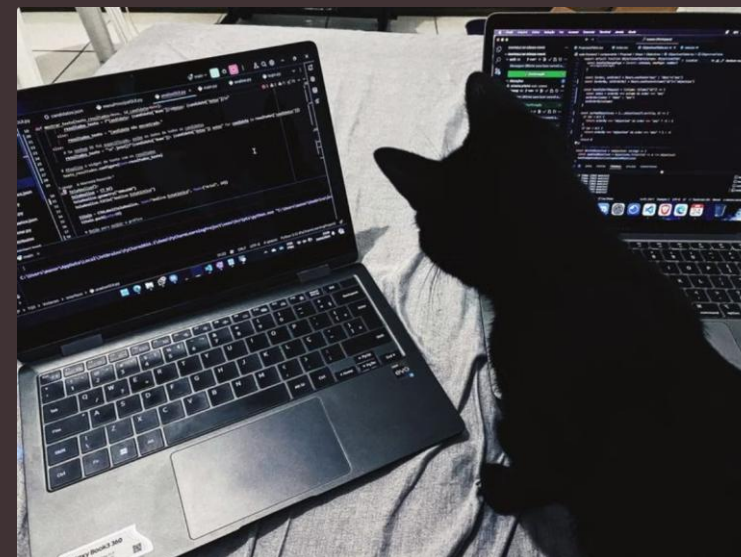
📌 La recomendación más importante es de orden actitudinal: cultivar el hábito de justificar cada decisión de diseño. Evaluar el código no solo por si funciona, sino por si está bien diseñado, es lo que diferencia a un programador competente de un ingeniero de software de excelencia.

Recomendaciones técnicas: persistencia con `json`, manejo de excepciones `try/except`, pruebas unitarias con `pytest`, y un modo inverso donde la computadora elige el número. Recomendaciones pedagógicas: programación orientada a objetos, medición experimental de rendimiento con `timeit`, interfaz gráfica con Tkinter, y estudio de colecciones especializadas del módulo `collections`.

Agradecimientos Adicionales

Este proyecto, desarrollado con mucha constancia y esfuerzo, ha sido una profunda inmersión en la lógica de programación y la aplicación de algoritmos. La implementación del juego "Adivina el Número" no solo consolidó conocimientos técnicos, sino que también reforzó la importancia de la autonomía en el aprendizaje y la resolución de problemas.

La dedicación personal en cada etapa, desde la conceptualización hasta la depuración, ha permitido explorar a fondo cada aspecto del desarrollo, comprendiendo la sinergia entre las estructuras de datos, los ciclos de control y la eficiencia algorítmica.



Reitero mi sincero agradecimiento a la **Ingeniera**, cuya orientación ha sido fundamental. Su experiencia y retroalimentación han transformado cada desafío en una valiosa oportunidad de crecimiento. Este trabajo es un testimonio de la dedicación y el aprendizaje continuo.

Expresando mi más sincera gratitud por la invaluable guía y el apoyo constante de la **Ingeniera** a lo largo de este proyecto. Su experiencia y retroalimentación fueron cruciales para la conceptualización, el desarrollo y la depuración del código, transformando este ejercicio en una experiencia de aprendizaje profundamente enriquecedora.